

Programming Language Principles, Design, and Implementation

Compilers - Register Allocation

Vincent Rahli

University of Birmingham

Where are we?

- ▶ Part 1: Principles of programming
- ▶ **Part 2: Compilers**

Today

Register allocation

- ▶ Linear scan register allocation
- ▶ Graph-coloring register allocation

Further reading:

- ▶ Chapter 8 of the Dragon book

Problem

Make efficient use of the memory hierarchy.

- ▶ Instructions involving registers are faster than using memory
- ▶ Small quantity of registers

Critical to make good use of the registers: reduce memory reads & writes.

Register allocation: deciding at each point in a program what values should reside in registers

Register assignment: in which register each value should reside

What do we do when we run out of registers?

Linear Scan Register Allocation

Advantages:

- ▶ very efficient algorithm
- ▶ often used in JIT (fast compilation)

Disadvantages:

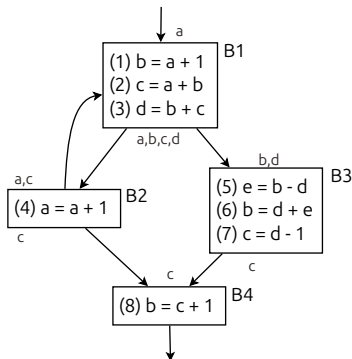
- ▶ imprecise
- ▶ other techniques lead to more efficient code

Linear Scan Register Allocation

Live variable: a variable is live immediately following an instruction if it will be used in the future, and won't be changed before that.

Live interval: for a variable, an interval such that the variable is not live outside the interval

For example:



Live intervals:

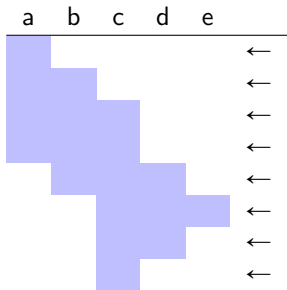
- ▶ a: [1-4]
- ▶ b: [2-5]
- ▶ c: [3-8]
- ▶ d: [5-7]
- ▶ e: [6-6]

Linear Scan Register Allocation

Register allocation algorithm:

- ▶ when a live interval begins: assign the variable to a free register
- ▶ when a live interval ends: the register is freed
- ▶ when there is no free register: ? – we will see later

Execute the algorithm

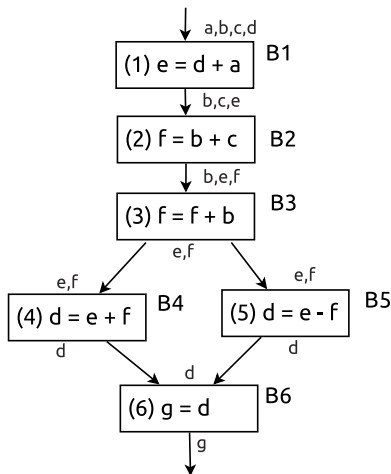


Registers:

R_1	R_2	R_3
ad	be	c

Linear Scan Register Allocation

Another example:



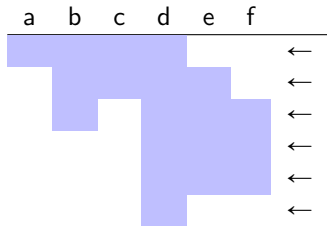
Live intervals:

- ▶ a: [1-1]
- ▶ b: [1-3]
- ▶ c: [1-2]
- ▶ d: [1-6]
- ▶ e: [2-5]
- ▶ f: [3-5]

Linear Scan Register Allocation

Execute the algorithm

Given 4 registers:

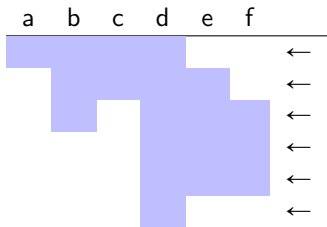


Registers:

R_1	R_2	R_3	R_4
ae	b	cf	d

Linear Scan Register Allocation

What if we have 3 registers only?



Registers:

R_1	R_2	R_3
ae	b	cf

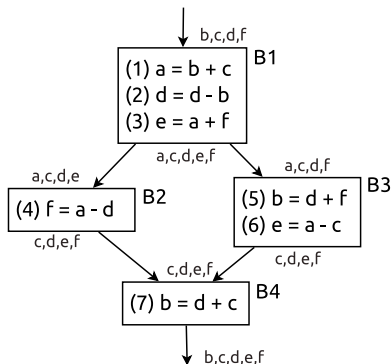
d is spilled (stored in memory)

Note: one strategy is to spill intervals that end last

Note: if a or c had been spilled, two variables would have spilled

Linear Scan Register Allocation

Another example:



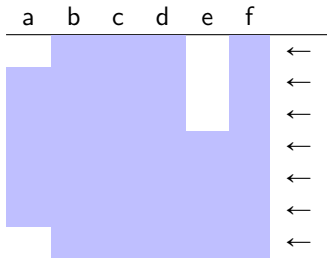
Live intervals:

- ▶ a: [2-6]
- ▶ b: [1-7]
- ▶ c: [1-7]
- ▶ d: [1-7]
- ▶ e: [4-7]
- ▶ f: [1-7]

Linear Scan Register Allocation

Execute the algorithm

Given 3 registers:



Registers:

R_1	R_2	R_3
b	c	da

f is spilled d and f are spilled d, e
and f are spilled

Graph-Coloring Register Allocation

Allocation problem reduced to a graph coloring problem:

- ▶ **Node**: a symbolic register
- ▶ **Edge**: between 2 variables live at the same program point

Goal:

- ▶ color the graph using R colors (the number of actual registers)
- ▶ such that two connected nodes get different colors

If a graph is not colorable, variables need to be spilled

Disadvantages:

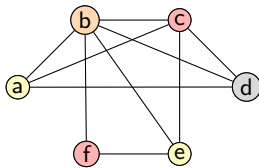
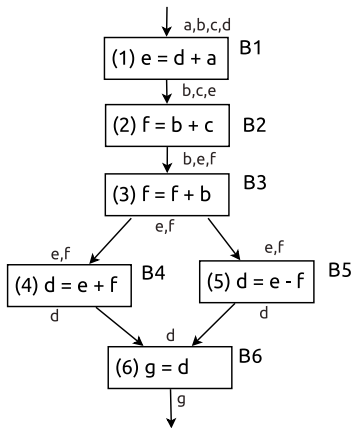
- ▶ the graph coloring problem is NP-complete

Advantages:

- ▶ there are heuristics that are often efficient in practice

Basis of the algorithm used by GCC

Graph-Coloring Register Allocation



If a node has less than R ($= 3$ here) white neighbors, color it
If some white nodes remain, a coloring is not possible & a node is spilled (grayed out)

Graph-Coloring Register Allocation

Chaitin's algorithm:

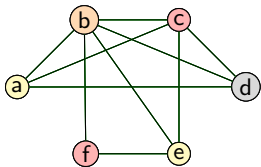
3 colors:

c1

c2

c3

graph:



stack:

a

b

c

d

e

f

General rule: avoid spills in inner loops

Graph-Coloring Register Allocation

Execute the algorithm

Another example:

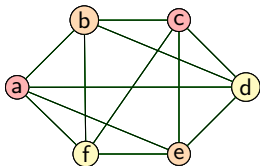
3 colors:

c1

c2

c3

graph:



stack:

d

f

e

b

c

a

We don't need to spill **c**: a color is available

We don't need to spill **a**: a color is available

If we had colored **f**, **e**, and **d** using different colors, we would have had to spill **c** and **a**

Chaitin-Briggs algorithm

Conclusion

What did we cover today?

- ▶ Register allocation
- ▶ Linear scan register allocation
- ▶ Graph-coloring register allocation